

```

import React, {
  forwardRef,
  useImperativeHandle,
  useRef,
  useState,
  useEffect,
} from "react";
import { Bold, Italic, Underline, List, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface HtmlMemoActionConfig {
  icon: LucideIcon;
  onClick: (htmlContent: string) => void;
  tooltipText?: string;
}

export interface HtmlMemoBoxProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  defaultValue?: string;
  onChange?: (html: string) => void;
  actionIcons?: HtmlMemoActionConfig[];
  disabled?: boolean;
  required?: boolean;
}

export interface HtmlMemoBoxRef {
  getHtml: () => string;
  setHtml: (html: string) => void;
  focus: () => void;
}

export const HtmlMemoBox = forwardRef<HtmlMemoBoxRef, HtmlMemoBoxProps>(
  (
    {
      label,
      error,
      defaultValue = "",
      onChange,
      actionIcons = [],
      disabled = false,
      className,
    }
  )

```

```

    required,
    ...props
  },
  ref,
) => {
  const editorRef = useRef<HTMLDivElement>(null);
  const toolbarRef = useRef<HTMLDivElement>(null);
  const [htmlContent, setHtmlContent] = useState<string>(defaultValue);
  const [isFocused, setIsFocused] = useState(false);
  const hasError = Boolean(error);

  useImperativeHandle(ref, () => ({
    getHtml: () => htmlContent,
    setHtml: (html: string) => {
      setHtmlContent(html);
      if (editorRef.current) {
        editorRef.current.innerHTML = html;
      }
      onChange?.(html);
    },
    focus: () => {
      editorRef.current?.focus();
    },
  })));

  useEffect(() => {
    if (editorRef.current && editorRef.current.innerHTML !== htmlContent) {
      editorRef.current.innerHTML = htmlContent;
    }
  }, [htmlContent]);

  const handleInput = () => {
    if (editorRef.current) {
      const html = editorRef.current.innerHTML;
      setHtmlContent(html);
      onChange?.(html);
    }
  };

  const handleFocus = () => setIsFocused(true);
  const handleBlur = () => setIsFocused(false);

  const executeCommand = (command: string, value?: string) => {
    document.execCommand(command, false, value);
    editorRef.current?.focus();
    handleInput();
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLDivElement>) => {
    if ((e.ctrlKey || e.metaKey) && e.key === "b") {

```

```

    e.preventDefault();
    executeCommand("bold");
  }
  if ((e.ctrlKey || e.metaKey) && e.key === "i") {
    e.preventDefault();
    executeCommand("italic");
  }
  if ((e.ctrlKey || e.metaKey) && e.key === "u") {
    e.preventDefault();
    executeCommand("underline");
  }
};

const limitedActionIcons = actionIcons.slice(0, 4);

const toolbarButtons = [
  { icon: Bold, command: "bold", tooltip: "Bold (Ctrl+B)" },
  { icon: Italic, command: "italic", tooltip: "Italic (Ctrl+I)" },
  { icon: Underline, command: "underline", tooltip: "Underline (Ctrl+U)" },
  { icon: List, command: "insertUnorderedList", tooltip: "Bullet List" },
];

return (
  <div className={cn("w-full", className)} {...props}>
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="relative">
      <div
        ref={toolbarRef}
        className={cn(
          "flex items-center gap-1 p-2",
          "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border-b",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-t-surface",
          disabled && "opacity-50 pointer-events-none",
        )}

```

```

role="toolbar"
aria-label="Text formatting"
>
{toolbarButtons.map((btn, index) => (
  <button
    key={index}
    type="button"
    className={cn(
      "relative flex items-center justify-center",
      "w-8 h-8 rounded-md",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-all duration-150 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
      "disabled:opacity-50 disabled:pointer-events-none",
    )}
    aria-label={btn.tooltip}
    disabled={disabled}
    onClick={() => executeCommand(btn.command)}
  >
    <btn.icon className="w-4 h-4" aria-hidden="true" />
  </button>
)}}
</div>
<div
  ref={editorRef}
  className={cn(
    "w-full bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
    "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
    "rounded-b-surface",
    "text-text-main placeholder:text-muted-foreground/60",
    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
    "disabled:opacity-50 disabled:pointer-events-none
disabled:cursor-not-allowed",
    "transition-all duration-200 ease-out",
    "px-4 py-3",
    "min-h-[120px]",
    "outline-none",
    hasError &&
      "border-destructive/50 focus-visible:ring-destructive/50",
    disabled && "bg-muted/50",
    isFocused &&
      "ring-2 ring-ring ring-offset-2 ring-offset-background",
  )}
  contentEditable={!disabled}
  suppressContentEditableWarning={true}
  onInput={handleInput}

```

```

onFocus={handleFocus}
onBlur={handleBlur}
onKeyDown={handleKeyDown}
role="textbox"
aria-multiline="true"
aria-label={label}
aria-invalid={hasError}
aria-describedby={error ? "html-memo-error" : undefined}
/>
{limitedActionIcons.length > 0 && (
  <div className="absolute bottom-2 right-2 flex items-center gap-1.5">
    {limitedActionIcons.map((action, index) => (
      <button
        key={index}
        type="button"
        className={cn(
          "relative flex items-center justify-center",
          "w-8 h-8 rounded-md",
          "text-muted-foreground/60 hover:text-foreground",
          "bg-transparent hover:bg-accent",
          "transition-all duration-150 ease-out",
          "active:scale-95",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
          "disabled:opacity-50 disabled:pointer-events-none",
        )}
        aria-label={action.tooltipText}
        disabled={disabled}
        onClick={() => action.onClick(htmlContent)}
      >
        <action.icon className="w-4 h-4" aria-hidden="true" />
      </button>
    ))}
  </div>
)}
</div>
{error && (
  <p
    id="html-memo-error"
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>

```

```
    );  
  },  
);
```

```
HtmlMemoBox.displayName = "HtmlMemoBox";
```